

Predicting load prices with CatBoost

Prepared 6 October 2026 | January-December 2025 data | Reproducible Python solution

The solution predicts a load's total **posted_rate** in US dollars. The final CatBoost pipeline was fitted on all 48,000 labeled January-October loads and produces 12,000 November-December predictions, plus the required 31-day fixed-route December scenario.

Input	Rows	Role
train-test.csv	48,000	Labeled development data; chronological training, dev and test periods.
validation.csv	12,000	Final inputs with hidden targets; matched to the template by load_id.
december-chart-inputs.csv	31	Fixed route and cargo; only the date changes.

Exploration and data quality

Distance is the strongest feature in the evaluated CatBoost model (92.3% of its reported feature importance), followed by equipment (3.6%). Importance describes this fitted model; it does not establish causality. Prices have a heavy error tail, so MAE and RMSE are both reported.

Issue	Observed counts	Treatment
Negative weight	292 labeled; 145 final-input rows	Apply abs() in memory and retain a weight_was_negative flag.
Missing weight	300 labeled; 165 final-input rows	Keep NaN for CatBoost and add weight_is_missing.
Missing market_index	374 labeled; 249 final-input rows	Exclude the signal from the shared pipeline; it is also absent from December inputs.

The weight investigation compared relationships with price, distance, equipment, routes and dates, including approximately matched loads. These checks support investigating a sign-entry error but do not prove every magnitude is correct. Absolute-value correction is an explicit modeling assumption adopted for this project, not a conclusion based on equal medians.

No labeled rows or target outliers were removed. Source CSVs are preserved byte-for-byte; cleaning occurs inside the pipeline. Negative longitudes are valid geographic values and are not changed.

Evaluate the future using the past

Partition	2025 dates	Rows	Purpose
Train	Jan 1 - Jun 30	28,806	Fit candidate models and preprocessing.
Development	Jul 1 - Aug 31	9,671	Select settings and early-stopping point.
Test / comparison	Sep 1 - Oct 31	9,523	Measure after refitting on Jan-Aug.

Date boundaries are deterministic. A seed does not choose the split. CatBoost, XGBoost and FFN all use seed 42 for training, and the alternative pipelines verify their exact load IDs against the saved CatBoost split manifest. Assertions reject overlap, omitted rows and non-chronological boundaries.

Preprocessing is learned only from the fitting rows: January-June during selection and January-August for local test evaluation. The test period is never an early-stopping input. After evaluation, the fixed final CatBoost configuration is refitted on all January-October labels; final-input data is not used to learn preprocessing.

Fourteen model inputs

Group	Features and purpose
Load and route	pickup, delivery and equipment categories; distance in miles; absolute weight in pounds. These describe the route, trailer and shipment size.
Data-quality flags	weight_is_missing and weight_was_negative preserve information about uncertain or corrected weights.
Geography	pickup_lat, pickup_lon, delivery_lat and delivery_lon remain separate numerical features. Missing coordinates use a city lookup learned from fitting rows.
Calendar	days_since_start, day_of_week and day_of_month describe time and recurring calendar patterns.

load_id is used only for matching outputs. posted_rate and predicted_rate are excluded from inputs. quote_signal is excluded because its price relationship changes across months. market_index is excluded because the December scenario does not provide it.

CatBoost accepts numeric NaNs and categorical strings directly. XGBoost and FFN instead use training-fitted median imputation and one-hot encoding; FFN also standardizes numeric inputs and its target. Predictions are returned in dollars with a fixed \$0.01 minimum.

Scope of the evidence

September-October was inspected during exploration and reused for model-family comparison. It is therefore not a fresh, untouched test of the final model choice. No unseen cities occur in the local evaluation periods, and November-December labels are hidden. The final refitted model has no independently measured final-period accuracy.

Measured results and model choice

Model	Dev MAE	Test MAE	Test RMSE	Test MAPE	Within 10%
Baseline	\$222.73	\$229.10	\$670.66	10.49%	67.38%
CatBoost	\$155.66	\$128.08	\$643.24	6.62%	94.90%
XGBoost	\$135.37	\$130.53	\$639.58	6.48%	93.80%
FFN	\$284.58	\$180.80	\$644.25	9.67%	77.76%

Dev: 9,671 July-August rows. Test/comparison: the same 9,523 September-October rows for all models. The baseline multiplies distance by the fitting data's median price per mile for each equipment type.

MAE is the average absolute dollar error; **RMSE** gives large errors more weight. **MAPE** averages absolute error divided by actual price. **Within 10%** is the share of loads whose relative absolute error is at most 0.10. This is a regression problem: 94.90% within 10% is not classification accuracy.

Why the submitted model is CatBoost

The CatBoost configuration was selected within its family using development MAE: depth 6, MAE loss and 94 trees, with learning rate 0.05 and seed 42. The initial search allowed 1,200 trees and stopped after 100 rounds without development improvement. Its categorical and missing-value handling keeps the deployed pipeline simple.

The final submission retains this established CatBoost pipeline. CatBoost has the lowest reused-test MAE, while XGBoost wins development MAE and slightly improves test RMSE and MAPE. These results do not establish that CatBoost is universally best or independently validate the final family choice. XGBoost selected depth 4 and 81 rounds; the FFN selected 128/64 hidden units and a two-epoch development checkpoint and did not improve on the tree pipelines.

Errors that still matter

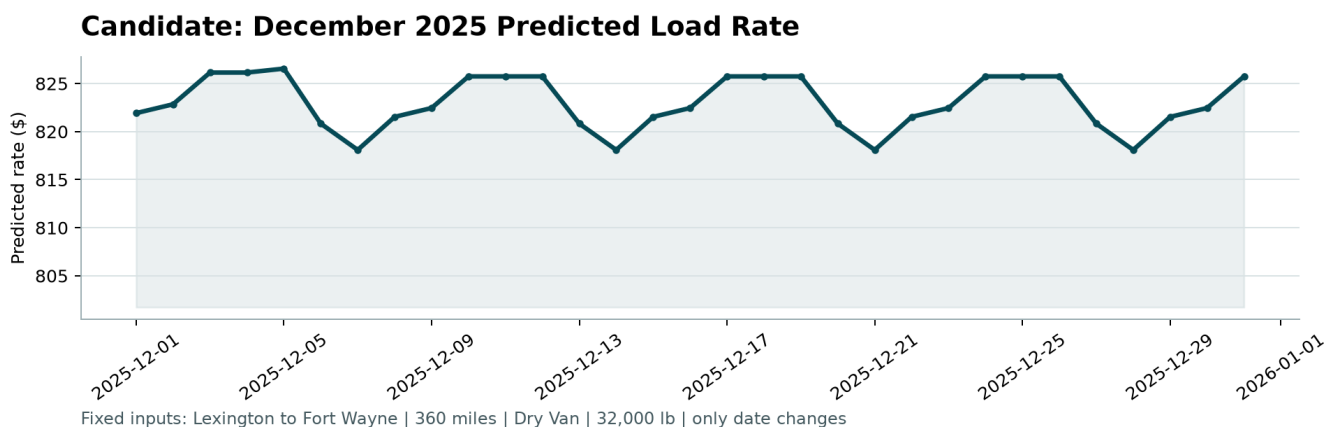
CatBoost reduces local test MAE by **44.1%** versus the baseline. Test R-squared is 0.8223; median absolute error is \$42.52. However, RMSE is \$643.24 and mean signed error is -\$67.04, indicating large-error cases and overall underprediction. In 128 of 9,523 test rows, absolute error exceeds \$1,000.

Longer routes remain harder in absolute dollars: routes above 1,500 miles have MAE \$232.64, compared with \$59.12 for 250-750 miles. No target outliers were removed to improve these scores. Model tuning used one fixed seed and a small candidate set; future work should use predeclared rolling time windows and a fresh evaluation period.

Evidence: [outputs/comparison/comparison_metrics.csv](#); [outputs/catboost/test_metrics.csv](#); [outputs/catboost/slice_metrics.csv](#); [outputs/catboost/feature_importance.csv](#).

The required December chart

The unchanged provided score.py generated this chart from december_predictions.csv. Every row represents Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb. Only the date varies from December 1 to December 31, 2025.



Predictions range from **\$818.09** to **\$826.55** across 9 distinct rounded prices. The vertical axis is zoomed, so the visible changes should be read relative to this small dollar range.

Absent coordinate columns are filled using the final model's fitted city lookup. Market and quote signals are not required. The same final model produces both submission files, without retraining or adding December assumptions.

How to interpret the shape

The variation reflects the model's available calendar features for a fixed shipment. Trees cannot extrapolate a new trend beyond their fitted ordinal dates. Training data ends in October and contains no labeled December history, so this chart is not evidence that holiday or year-end seasonality has been learned. No artificial holiday premium or trend was added.

Official checks passed

The scorer accepted all 12,000 validation IDs and positive finite predictions. It also accepted all 31 December dates, the original seven-column order, the fixed shipment attributes and positive predictions. Its output states that final validation metrics are calculated by Spotter after submission.

The scorer validates submission structure and draws the chart; it does not measure prediction quality against hidden targets. Its complete output is retained in scorer_results/scorer_output.txt.

Run, inspect and reproduce

Repository: github.com/Mutasem-Tamimi/spotter

Artifact	Contents
validation_predictions.csv	Required underscore filename; 12,000 rows, load_id and predicted_rate, template order.
december_predictions.csv	Completed 31-row scenario; supplied input file preserved.
scorer_results/candidate_december.png	Required chart generated by the original score.py.
notebooks/01 through 05	Exploration, CatBoost, alternatives, final refit, and executed submission checks/report.
outputs/final_catboost/	Full-data prediction pipeline and metadata; older evaluation artifacts remain separate.
outputs/submission/submission_checks.json	Input/model/output hashes, schema checks and final prediction audit.

Rebuild on Windows with Python 3.13

Run these commands from the repository root. Activation is optional.

```
py -3.13 -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe -m pip install -r requirements-report.txt
.venv\Scripts\python.exe predict_validation.py
.venv\Scripts\python.exe complete_submission.py
.venv\Scripts\python.exe build_report.py
.venv\Scripts\python.exe -m unittest discover -s tests -v
```

predict_validation.py refits the saved, fixed CatBoost settings on all labeled rows. complete_submission.py loads that final pipeline, predicts both schemas and runs score.py. build_report.py rebuilds this PDF from saved comparison results and the official chart. To retrain the local evaluation experiments, run freight_model.py, xgboost_experiment.py and ffn_experiment.py, or follow the notebook instructions.

Checks and remaining scientific limitations

The submission workflow verifies IDs, order, row counts, numeric values and rounding after CSV reload. Hash checks confirm that source CSVs and readme-spotter.md remain unchanged. Earlier evaluation artifacts are preserved separately from the full-data refit.

All reported performance belongs to local chronological evaluation. Reused test-period selection, unmeasured unseen-city behavior, uncertain weight corrections, large-error tails and unavailable final targets limit what can be claimed. A subsequent improvement cycle should establish fresh rolling-origin validation before tuning further.